

FactFox

Days 1 to 4 Beginner Guide

A beginner-friendly explanation of what we built, why we built it, and what every important line of code does.

Project: FactFox - a browser extension that scans prompts and AI-generated text for malicious or suspicious intent.

Audience: beginner with basic HTML, CSS, JavaScript, Git, and web fundamentals.

1. What We Built In The First Four Days

During the first four days, we moved FactFox from a simple project idea into a working browser extension. The extension currently opens as a Chrome popup, accepts text from the user, scans it, and shows a safety result with a score, category, reason, and risk meter.

Day	What we built	Why it matters
Day 1	Created project folder, GitHub repo, Streamlit prototype, and this guide to the project files.	This gave the project structure, version control, and a working starting point.
Day 2	Added better risk scoring and UI wording.	The extension became more than a basic keyword checker.
Day 3	Added a risk meter, score badge, result category, and improved the result layout.	The result now looks like a product report.
Day 4	Added more categories, Clear button, README updates, and the project GitHub.	The project became cleaner, better documented, and safely backed up.

2. Project Folder Structure

A good project is not only code. It also has folders that make the project easy to understand, maintain, and show on GitHub.

```
factfox/  
  app/  
    main.py  
  assets/  
    logo.png  
  data/  
    sample_prompts.csv  
  docs/  
    FactFox_Days_1_to_4_Beginner_Guide.pdf  
  extension/  
    manifest.json  
    popup.html  
    popup.css  
    popup.js  
  models/  
  notebooks/  
  README.md  
  requirements.txt  
  .gitignore
```

Why each folder exists

- app: contains the early Streamlit prototype. This was our first simple demo.
- assets: stores branding files such as the FactFox logo.
- data: stores example prompts and future training data.
- docs: stores documentation like this PDF guide.
- extension: contains the actual Chrome extension code.
- models: reserved for future machine learning models.
- notebooks: reserved for future data science experiments.

- README.md: the GitHub front page for recruiters and developers.
- requirements.txt: Python dependencies for the Streamlit prototype.
- .gitignore: tells Git which files should not be uploaded.

3. How A Chrome Extension Works

A browser extension is a small app that runs inside the browser. For FactFox, Chrome reads manifest.json first. The manifest tells Chrome the extension name, version, popup file, and permissions. Then popup.html creates the visible popup, popup.css styles it, and popup.js makes it interactive.

```
manifest.json -> tells Chrome what the extension is
popup.html    -> creates the popup structure
popup.css     -> controls how the popup looks
popup.js      -> controls how the popup behaves
```

4. manifest.json Explained

```
{
  "manifest_version": 3,
  "name": "FactFox",
  "version": "1.0.0",
  "description": "Detect malicious prompts and verify AI-generated responses.",
  "action": {
    "default_popup": "popup.html",
    "default_title": "FactFox"
  },
  "permissions": ["activeTab"]
}
```

Line-by-line explanation

- manifest_version: 3 means we are using Chrome's modern extension format.
- name: FactFox is the extension name shown in Chrome.
- version: 1.0.0 is the current extension version number.
- description explains what the extension does in one short sentence.
- action means this extension has a toolbar button that can open a popup.
- default_popup points Chrome to popup.html when the user clicks FactFox.
- default_title is the tooltip text shown when hovering over the extension icon.
- permissions: activeTab allows the extension to interact with the current tab later. We added it because FactFox will eventually scan selected text and page text.

5. popup.html Explained

```
<!DOCTYPE html>
<html>
  <head>
    <title>FactFox</title>
    <link rel="stylesheet" href="popup.css">
  </head>

  <body>
    <div class="container">
      <h1>FactFox</h1>
      <p class="subtitle">Prompt risk and AI response safety scanner</p>

      <textarea
        id="promptInput"
        placeholder="Paste a prompt, AI answer, or selected text to scan..."
      ></textarea>

      <p class="helper">
        FactFox checks for prompt injection, credential theft, destructive actions, and malware intent.
      </p>

      <button id="analyzeBtn">Analyze</button>
      <button id="clearBtn" class="secondary">Clear</button>

      <div id="resultBox" class="result hidden">
        <div class="scoreRow">
          <h2 id="resultLabel"></h2>
          <span id="scoreBadge"></span>
        </div>

        <div class="meter">
          <div id="meterFill"></div>
        </div>

        <p id="resultCategory"></p>
        <p id="resultReason"></p>
      </div>

      <script src="popup.js"></script>
    </body>
  </html>
```

Main idea

HTML is the skeleton of the popup. It decides which elements exist: title, subtitle, text box, buttons, result area, score badge, meter, category, and reason.

Important lines

- tells the browser this is an HTML5 document.
- wraps the whole page.
- contains information about the page, not the visible content.
- FactFox sets the popup page title.
- The stylesheet link connects popup.css so the popup has styling.
- contains everything the user sees.
- The container div groups the whole popup content so we can style it together.
- FactFox shows the project name.
- The subtitle explains the purpose of the extension in one line.
- textarea creates the box where the user pastes a prompt or AI answer.
- id='promptInput' is important because JavaScript uses that ID to read the typed text.
- The helper paragraph tells the user what kinds of risks FactFox checks.

- analyzeBtn creates the Analyze button. JavaScript listens for clicks on this button.
- clearBtn creates the Clear button. JavaScript uses it to reset the popup.
- resultBox is where the scan result appears.
- class='result hidden' means it uses result styling but starts hidden.
- scoreRow keeps the label and score badge on the same row.
- scoreBadge shows values such as 95/100.
- meter and meterFill create the progress bar style risk meter.
- resultCategory shows categories like Credential Theft or Prompt Injection.
- resultReason explains why FactFox made that decision.
- The script tag loads popup.js at the end so HTML elements exist before JavaScript tries to use them.

6. popup.css Explained

CSS controls the design: colors, sizes, spacing, borders, button styles, and hidden/show behavior.

```
body {
  margin: 0;
  font-family: Arial, sans-serif;
  background: #101418;
  color: #f5f5f5;
  width: 360px;
}

.container {
  padding: 18px;
}

h1 {
  margin: 0;
  color: #ff7a1a;
  font-size: 28px;
}

.subtitle {
  margin-top: 4px;
  color: #8fbbe8;
  font-size: 14px;
}

.helper {
  margin: 10px 0 0;
  color: #b8c7d9;
  font-size: 12px;
  line-height: 1.4;
}

textarea {
  width: 100%;
  height: 120px;
  margin-top: 16px;
  padding: 10px;
  box-sizing: border-box;
  resize: none;
  border: 1px solid #2d3742;
  border-radius: 8px;
  background: #171d24;
  color: #ffffff;
  font-size: 14px;
}

button {
  width: 100%;
  margin-top: 12px;
  padding: 10px;
  border: none;
  border-radius: 8px;
  background: #ff7a1a;
  color: #101418;
  font-weight: bold;
}
```

```

    cursor: pointer;
}

button:hover {
    background: #ff9345;
}

.result {
    margin-top: 14px;
    padding: 12px;
    border-radius: 8px;
    background: #171d24;
    border-left: 4px solid #8fbbe8;
}

.result h2 {
    margin: 0 0 6px;
    font-size: 18px;
}

.result p {
    margin: 0;
    color: #d6d6d6;
}

.hidden {
    display: none;
}

.safe {
    border-left-color: #2ecc71;
}

.suspicious {
    border-left-color: #f1c40f;
}

.malicious {
    border-left-color: #e74c3c;
}

.scoreRow {
    display: flex;
    align-items: center;
    justify-content: space-between;
    gap: 8px;
}

#scoreBadge {
    padding: 4px 8px;
    border-radius: 999px;
    background: #243241;
    color: #ffffff;
    font-size: 12px;
    font-weight: bold;
}

.meter {
    width: 100%;
    height: 8px;
    margin: 8px 0 10px;
    border-radius: 999px;
    background: #26313d;
    overflow: hidden;
}

#meterFill {
    width: 0%;
    height: 100%;
    border-radius: 999px;
    background: #2ecc71;
    transition: width 0.25s ease;
}

#resultCategory {
    margin: 0 0 6px;
    color: #8fbbe8;
    font-weight: bold;
}

#resultReason {
    line-height: 1.4;
}

```

```

}

.secondary {
  margin-top: 8px;
  background: #243241;
  color: #f5f5f5;
}

.secondary:hover {
  background: #304357;
}

```

Important styling decisions

- body sets the popup width, background color, text color, and font.
- margin: 0 removes the browser's default empty spacing around the page.
- width: 360px gives the extension popup a stable size.
- .container adds padding so content does not touch the popup edges.
- h1 uses orange to match the FactFox branding.
- .subtitle uses blue to create a visual identity with orange and blue.
- .helper is smaller text that explains the scanner without dominating the UI.
- textarea styles the text input box with a dark background, border, padding, and fixed height.
- box-sizing: border-box makes width calculations easier because padding stays inside the 100 percent width.
- button styles both buttons with full width and strong color.
- button:hover changes the button color when the user moves the mouse over it.
- .result creates the card that displays the scan output.
- .hidden uses display: none so the result area is invisible before analysis.
- .safe, .suspicious, and .malicious change the left border color based on result level.
- .scoreRow uses flexbox to place the result label and score badge in one row.
- #scoreBadge uses a pill shape to make the numeric score easy to notice.
- .meter is the background track for the risk meter.
- #meterFill is the colored part of the meter. JavaScript changes its width based on the risk score.
- #resultCategory is bold and blue because it is a key part of the report.
- .secondary styles the Clear button differently from the main Analyze button.

7. popup.js Explained

JavaScript makes FactFox interactive. It reads what the user typed, checks it against risk rules, calculates the highest risk, and updates the popup UI.

```
const promptInput = document.getElementById("promptInput");
const analyzeBtn = document.getElementById("analyzeBtn");
const clearBtn = document.getElementById("clearBtn");
const resultBox = document.getElementById("resultBox");
const resultLabel = document.getElementById("resultLabel");
const resultReason = document.getElementById("resultReason");
const scoreBadge = document.getElementById("scoreBadge");
const meterFill = document.getElementById("meterFill");
const resultCategory = document.getElementById("resultCategory");

const riskRules = [
  {
    category: "Destructive Action",
    score: 90,
    level: "Malicious",
    patterns: [
      "delete all files",
      "wipe my computer",
      "format my drive",
      "erase everything",
    ],
    reason: "The prompt asks for destructive actions that could damage files or systems.",
  },
  {
    category: "Credential Theft",
    score: 95,
    level: "Malicious",
    patterns: [
      "steal password",
      "steal passwords",
      "steal saved passwords",
      "get saved passwords",
      "extract login details",
      "recover saved passwords",
      "dump passwords",
    ],
    reason: "The prompt attempts to access or steal private credentials.",
  },
  {
    category: "API Key Leakage",
    score: 90,
    level: "Malicious",
    patterns: [
      "send my api key",
      "send my api keys",
      "leak api key",
      "expose api key",
    ],
    reason: "The prompt asks to expose sensitive API credentials.",
  },
  {
    category: "Data Exfiltration",
    score: 88,
    level: "Malicious",
    patterns: [
      "send confidential data",
      "upload private files",
      "exfiltrate data",
      "send files to another server",
      "copy private documents"
    ],
    reason: "The prompt attempts to move private or confidential data outside the user's control."
  },
  {
    category: "Prompt Injection",
    score: 70,
    level: "Suspicious",
    patterns: [
      "ignore previous instructions",
      "reveal your system prompt",
      "bypass safety",
      "disable restrictions",
    ],
    reason: "The prompt attempts to override or bypass AI safety instructions.",
  },
];
```

```

},
{
  category: "Jailbreak Attempt",
  score: 65,
  level: "Suspicious",
  patterns: [
    "pretend you are not an ai",
    "jailbreak",
    "act without rules",
    "ignore your rules",
  ],
  reason:
    "The prompt appears to be trying to bypass normal AI behavior limits.",
},
{
  category: "Malware Or Abuse",
  score: 85,
  level: "Malicious",
  patterns: [
    "install malware",
    "create a virus",
    "write ransomware",
    "build a keylogger",
  ],
  reason: "The prompt requests harmful software or cyber abuse.",
},
{
  category: "Unsafe Automation",
  score: 75,
  level: "Suspicious",
  patterns: [
    "run this command without asking",
    "execute this silently",
    "do not ask for confirmation",
    "perform this action automatically"
  ],
  reason: "The prompt tries to make the AI perform actions without user confirmation."
},
];

function analyzeText(text) {
  const lowerText = text.toLowerCase();
  const matches = [];

  for (const rule of riskRules) {
    for (const pattern of rule.patterns) {
      if (lowerText.includes(pattern)) {
        matches.push({
          category: rule.category,
          score: rule.score,
          level: rule.level,
          pattern,
          reason: rule.reason,
        });
      }
    }
  }

  if (matches.length === 0) {
    return {
      label: "Safe",
      score: 10,
      category: "No Risk Detected",
      reason:
        "No obvious malicious, unsafe, or prompt injection pattern was detected.",
    };
  }

  matches.sort((a, b) => b.score - a.score);
  const topMatch = matches[0];

  return {
    label: topMatch.level,
    score: topMatch.score,
    category: topMatch.category,
    reason: `${topMatch.reason} Detected phrase: "${topMatch.pattern}".`,
  };
}

analyzeBtn.addEventListener("click", () => {
  const text = promptInput.value.trim();

```



```

if (!text) {
  resultBox.className = "result suspicious";
  resultLabel.textContent = "No input";
  scoreBadge.textContent = "0/100";
  meterFill.style.width = "0%";
  meterFill.style.background = "#f1c40f";
  resultCategory.textContent = "Input Required";
  resultReason.textContent = "Paste a prompt or AI answer first.";
  return;
}

const result = analyzeText(text);

resultBox.className = `result ${result.label.toLowerCase()}`;
resultLabel.textContent = result.label;
scoreBadge.textContent = `${result.score}/100`;
meterFill.style.width = `${result.score}%`;
resultCategory.textContent = result.category;
resultReason.textContent = result.reason;

if (result.score >= 80) {
  meterFill.style.background = "#e74c3c";
} else if (result.score >= 40) {
  meterFill.style.background = "#f1c40f";
} else {
  meterFill.style.background = "#2ecc71";
}
});
clearBtn.addEventListener("click", () => {
  promptInput.value = "";
  resultBox.className = "result hidden";
  resultLabel.textContent = "";
  scoreBadge.textContent = "";
  meterFill.style.width = "0%";
  resultCategory.textContent = "";
  resultReason.textContent = "";
});

```

Part 1: Connecting JavaScript to HTML

```

const promptInput = document.getElementById("promptInput");
const analyzeBtn = document.getElementById("analyzeBtn");
const clearBtn = document.getElementById("clearBtn");
const resultBox = document.getElementById("resultBox");
const resultLabel = document.getElementById("resultLabel");
const resultReason = document.getElementById("resultReason");
const scoreBadge = document.getElementById("scoreBadge");
const meterFill = document.getElementById("meterFill");
const resultCategory = document.getElementById("resultCategory");

```

Each line finds one HTML element by its ID and stores it in a variable. This lets JavaScript read from it or change it later. For example, promptInput lets JavaScript read the text area. resultLabel lets JavaScript write Safe, Suspicious, or Malicious into the result card.

Part 2: Risk rules

riskRules is an array of rule objects. An array is a list. Each object describes one risk category. This design is beginner-friendly and easy to expand.

```

const riskRules = [
  {
    category: "Credential Theft",
    score: 95,
    level: "Malicious",
    patterns: ["steal password", "steal saved passwords"],
    reason: "The prompt attempts to access or steal private credentials."
  }
];

```

- category is the type of risk.
- score is how dangerous the risk is from 0 to 100.
- level is the final label shown to the user.
- patterns are phrases FactFox searches for.
- reason is the explanation shown in the result card.

Part 3: analyzeText function

```
function analyzeText(text) {
  const lowerText = text.toLowerCase();
  const matches = [];

  for (const rule of riskRules) {
    for (const pattern of rule.patterns) {
      if (lowerText.includes(pattern)) {
        matches.push({ ... });
      }
    }
  }

  if (matches.length === 0) {
    return { label: "Safe", score: 10, ... };
  }

  matches.sort((a, b) => b.score - a.score);
  const topMatch = matches[0];
  return { ... };
}
```

- function analyzeText(text) creates reusable logic. We can call it whenever we want to scan text.
- text.toLowerCase() makes matching case-insensitive. This means 'Create a virus' and 'create a virus' both work.
- matches starts as an empty list where detected risks are stored.
- The first for loop checks each risk category.
- The second for loop checks each phrase inside that category.
- lowerText.includes(pattern) checks whether the user's text contains that risky phrase.
- matches.push stores the detected category, score, level, phrase, and reason.
- If no matches are found, FactFox returns Safe with a low score of 10.
- matches.sort orders detected risks from highest score to lowest score.
- topMatch becomes the most dangerous detected risk.
- The function returns the final result object used by the UI.

Part 4: Analyze button click

```
analyzeBtn.addEventListener("click", () => {
  const text = promptInput.value.trim();
  const result = analyzeText(text);
  resultBox.className = `result ${result.label.toLowerCase()}`;
  resultLabel.textContent = result.label;
  scoreBadge.textContent = `${result.score}/100`;
  meterFill.style.width = `${result.score}%`;
});
```

- addEventListener tells the browser to run this code when Analyze is clicked.
- promptInput.value reads what the user typed.
- trim removes extra spaces at the beginning and end.
- The no-input check handles the case where the user clicks Analyze without entering anything.
- analyzeText(text) runs the safety logic.
- className changes the result card color by applying safe, suspicious, or malicious CSS classes.
- textContent changes visible text inside the popup.
- meterFill.style.width changes the risk meter length. A score of 95 becomes 95 percent width.
- The if/else block changes the meter color: red for high risk, yellow for medium risk, green for low risk.

Part 5: Clear button click

```
clearBtn.addEventListener("click", () => {
  promptInput.value = "";
  resultBox.className = "result hidden";
  resultLabel.textContent = "";
  scoreBadge.textContent = "";
  meterFill.style.width = "0%";
  resultCategory.textContent = "";
  resultReason.textContent = "";
});
```

- This listens for a click on the Clear button.
- promptInput.value = empty string clears the text box.
- resultBox.className = result hidden hides the result card again.
- The remaining lines clear old result text and reset the meter.

8. Why This Is Still A Beginner Version

The current extension uses rule-based matching. That means it looks for specific phrases. This is good for a first version because it is simple, understandable, and easy to debug. But it can miss similar wording if the exact phrase is not in the rules. For example, we had to add 'steal saved passwords' because the first version did not catch it.

Strengths

- Fast and works offline.
- Easy to understand and explain to recruiters.
- Easy to add new categories and phrases.
- Good foundation for a browser extension.

Limitations

- It can miss risky text if the phrase is worded differently.
- It does not understand context deeply yet.
- It does not scan live typing yet.
- It does not verify factual claims yet.

9. How This Becomes Grammarly-Style

The long-term goal is for FactFox to behave like Grammarly for AI safety. Instead of only scanning text pasted into a popup, it should watch text boxes on pages and show warnings while the user types.

Current version:

User clicks extension -> pastes text -> clicks Analyze -> sees result

Future version:

User types on ChatGPT/Gemini/Claude -> FactFox scans live -> warning badge appears near textbox

Next technical pieces

- content.js: code that runs inside webpages.
- Selected text scanning: user highlights text and FactFox analyzes it.
- Textbox detection: FactFox finds textarea and input elements on pages.
- Live scanning: FactFox listens while the user types.
- Inline badge: FactFox shows Safe, Suspicious, or Malicious near the text box.
- Answer verification: FactFox detects claims in AI answers and marks them as needing evidence.

10. GitHub And Saving Work

We used Git and GitHub so every important version of the project is saved. Git saves versions on your laptop. GitHub stores the project online.

```
git status
# Shows what changed

git add .
# Prepares changed files for saving

git commit -m "message"
# Saves a named version locally

git push
# Uploads commits to GitHub
```

A clean project should usually end the day with git status showing: nothing to commit, working tree clean.

11. What You Can Say About The First Four Days

I built the first version of FactFox as a Chrome extension. It scans user-entered prompts or AI-generated text, detects risky patterns such as prompt injection, credential theft, malware intent, destructive actions, API key leakage, and unsafe automation, then displays a risk score, category, reason, and visual risk meter. The current version is rule-based, and the next version will move toward live webpage scanning like Grammarly.

12. Next Development Goal

The next major milestone is moving from popup scanning to page scanning. That means adding a content.js file and making FactFox read selected text or typed text from webpages. This is the first real step toward the Grammarly-style product vision.